YUN PERFUME — Production Task List

> **Last updated:** 2026-04-28
> **Stack:** Next.js 16 · TypeScript · Tailwind CSS v4 · GSAP + Lenis · Cloudflare Pages · Cloudflare D1 (SQLite) · Cloudflare R2 · Cloudflare KV · Drizzle ORM · Razorpay · Resend
> **Package manager:** Bun (primary)
> **Status:** Frontend ~90% done. Backend integrations ~10% done. Nothing is wired end-to-end.

CRITICAL DECISIONS (resolve before touching any code)

- [ ] **D1 vs Neon:** `docs/INFRASTRUCTURE.md` says D1. The codebase uses Neon/Prisma. **Decision: migrate to Cloudflare D1 + Drizzle ORM.** Prisma with D1 is experimental and not production-safe. Drizzle has first-class D1 support. All tasks below assume D1 + Drizzle.
- [ ] **wrangler.jsonc → wrangler.toml:** Rename and fix the config file (see Phase 0).
- [ ] **`pages_build_output_dir`:** Currently `.next` — this is wrong for Cloudflare Pages. Must be `.vercel/output/static` after `@cloudflare/next-on-pages` build.

PHASE 0 — Infrastructure & Config Fixes

> Fix all the broken plumbing before writing any feature code.

0.1 Replace Prisma + Neon with Drizzle + D1

- [ ] Install Drizzle and D1 dependencies:

```bash
bun add drizzle-orm
bun add -d drizzle-kit
```

- [ ] Remove Prisma:

```bash
bun remove prisma @prisma/client @prisma/adapter-neon @neondatabase/serverless
```

- [ ] Delete `prisma/` folder (back up `prisma/schema.prisma` first for reference — the schema design is good)
- [ ] Delete `src/lib/db.ts` (Prisma client) — will be replaced with Drizzle client in Phase 1
- [ ] Remove `DATABASE_URL` from `.env` (D1 has no connection string — it's a binding)

0.2 Install remaining dependencies

- [ ] Install all required packages:

```bash
bun add @cloudflare/next-on-pages
bun add razorpay
bun add resend
bun add zod
bun add jose
bun add nanoid
bun add -d wrangler
```

0.3 Fix wrangler config

- [ ] Delete `wrangler.jsonc`
- [ ] Create `wrangler.toml` at project root:

```toml
name = "yun-perfume"
compatibility_date = "2024-09-23"
compatibility_flags = ["nodejs_compat"]
pages_build_output_dir = ".vercel/output/static"

[[d1_databases]]
binding = "DB"
database_name = "yun-perfume-db"
database_id = "REPLACE_AFTER_STEP_2.1"

[[r2_buckets]]
binding = "IMAGES"
bucket_name = "yun-perfume-images"

[[kv_namespaces]]
binding = "SESSIONS"
id = "REPLACE_AFTER_STEP_2.3"
preview_id = "REPLACE_AFTER_STEP_2.3"

[vars]
NODE_ENV = "production"
NEXT_PUBLIC_APP_URL = "https://yunperfume.com"
```

> Note: `IMAGES` binding matches the existing `wrangler.jsonc` — keep it consistent.

0.4 Fix next.config

- [ ] Update `next.config.ts`:

```ts
import { setupDevPlatform } from '@cloudflare/next-on-pages/next-dev';

if (process.env.NODE_ENV === 'development') {
  await setupDevPlatform();
}

const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: '*.r2.cloudflarestorage.com' },
      { protocol: 'https', hostname: 'pub-*.r2.dev' },
    ],
  },
};

export default nextConfig;
```

0.5 Fix package.json scripts

- [ ] Update scripts in `package.json`:

```json
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "pages:build": "npx @cloudflare/next-on-pages",
  "deploy": "bun run pages:build && wrangler pages deploy",
  "db:generate": "drizzle-kit generate",
  "db:migrate:local": "wrangler d1 migrations apply yun-perfume-db --local",
  "db:migrate:remote": "wrangler d1 migrations apply yun-perfume-db --remote",
  "db:studio": "drizzle-kit studio",
  "db:seed": "bun scripts/seed.ts"
}
```

0.6 Fix .env.local

- [ ] Rename `.env` to `.env.local` (Next.js convention)
- [ ] Update `.env.local` to final shape:

```env
```

```
# Razorpay — use TEST keys during development
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxxxxxxxxxx
NEXT_PUBLIC_RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxx

# Resend
RESEND_API_KEY=re_xxxxxxxxxxxx

# Auth
JWT_SECRET=run: openssl rand -base64 64

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000

# R2 public URL (set after Phase 2.2)
NEXT_PUBLIC_R2_PUBLIC_URL=https://pub-XXXX.r2.dev
```

- [ ] Remove `DATABASE_URL` — no longer needed with D1
- [ ] Confirm `.env.local` is in `.gitignore`
- [ ] Create `.env.example` with all keys but empty values (safe to commit)

0.7 Add Cloudflare runtime to all API routes

- [ ] Every file in `src/app/api/` must have this at the top:

```ts
export const runtime = 'edge';
```

- [ ] Add to existing `src/app/api/auth/magic-link/route.ts`
- [ ] Add to existing `src/app/api/payment/create-order/route.ts`

0.8 Fix the shop page Server Component issue

- [ ] Open `src/app/shop/page.tsx`
- [ ] Remove the `useCart` import on line 5 — unused and will crash a Server Component build
- [ ] Confirm no other client hooks (`useState`, `useEffect`, etc.) exist in this file
- [ ] If client interactions are needed, split into a Server Component wrapper + `'use client'` child component

---

PHASE 1 — Database: Drizzle + D1

1.1 Create folder structure

- [ ] Create `src/db/` folder
- [ ] Create `src/db/schema.ts`
- [ ] Create `src/db/index.ts`
- [ ] Create `drizzle.config.ts` at project root

1.2 Write schema (mirrors existing Prisma schema exactly)

- [ ] Create `src/db/schema.ts`:

```ts
```

```
import { sqliteTable, text, integer } from 'drizzle-orm/sqlite-core';
import { sql } from 'drizzle-orm';

export const users = sqliteTable('users', {
  id: text('id').primaryKey(),
  email: text('email').notNull().unique(),
  name: text('name'),
  role: text('role').notNull().default('USER'), // 'USER' | 'ADMIN'
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
  lastLoginAt: text('last_login_at'),
});

export const collections = sqliteTable('collections', {
  id: text('id').primaryKey(),
  slug: text('slug').notNull().unique(),
  name: text('name').notNull(),
  description: text('description'),
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
});

export const products = sqliteTable('products', {
  id: text('id').primaryKey(),
  slug: text('slug').notNull().unique(),
  name: text('name').notNull(),
  tagline: text('tagline'),
  description: text('description').notNull(),
  price: integer('price').notNull(),           // paise: ₹4800 = 480000
  stock: integer('stock').notNull().default(0),
  active: integer('active', { mode: 'boolean' }).default(true),
  imageUrl: text('image_url'),                 // primary R2 URL
  images: text('images'),                      // JSON array of R2 URLs
  collectionId: text('collection_id').references(() => collections.id),
  scentFamily: text('scent_family'),           // e.g. "Floral · Warm"
  scentNotes: text('scent_notes'),             // JSON array: ["mogra","sandalwood
  concentration: text('concentration'),
  size: text('size').default('50ml'),
  isDiscoverySet: integer('is_discovery_set', { mode: 'boolean' }).default(false),
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
  updatedAt: text('updated_at').default(sql`CURRENT_TIMESTAMP`),
});

export const magicTokens = sqliteTable('magic_tokens', {
  id: text('id').primaryKey(),
  email: text('email').notNull(),
  token: text('token').notNull().unique(),
  expiresAt: text('expires_at').notNull(),
```

```ts
  usedAt: text('used_at'),
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
});

export const orders = sqliteTable('orders', {
  id: text('id').primaryKey(),                    // YUN-{timestamp}-{nanoid}
  userId: text('user_id').references(() => users.id),
  email: text('email').notNull(),
  status: text('status').notNull().default('PENDING'),
  // 'PENDING' | 'PAID' | 'SHIPPED' | 'DELIVERED' | 'CANCELLED' | 'FAILED'
  totalAmount: integer('total_amount').notNull(), // paise
  razorpayOrderId: text('razorpay_order_id'),
  razorpayPaymentId: text('razorpay_payment_id'),
  shippingAddress: text('shipping_address'),   // JSON string
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
  paidAt: text('paid_at'),
});

export const orderItems = sqliteTable('order_items', {
  id: text('id').primaryKey(),
  orderId: text('order_id').references(() => orders.id),
  productId: text('product_id').references(() => products.id),
  productName: text('product_name').notNull(), // snapshot at purchase time
  quantity: integer('quantity').notNull(),
  unitPrice: integer('unit_price').notNull(),  // paise
});
```

1.3 DB connection helper

- [ ] Create `src/db/index.ts`:

```ts
import { drizzle } from 'drizzle-orm/d1';
import * as schema from './schema';

export function getDb(d1: D1Database) {
  return drizzle(d1, { schema });
}

export type Database = ReturnType<typeof getDb>;
export * from './schema';
```

1.4 Drizzle config

- [ ] Create `drizzle.config.ts`:

```ts
import type { Config } from 'drizzle-kit';

export default {
  schema: './src/db/schema.ts',
  out: './migrations',
  dialect: 'sqlite',
  driver: 'd1-http',
} satisfies Config;
```

1.5 Generate migrations

- [ ] Run: `bun run db:generate`
- [ ] Verify `migrations/` folder is created with `.sql` files
- [ ] Commit migrations to git

1.6 Create D1 database and apply migrations

- [ ] Run: `wrangler d1 create yun-perfume-db`
- [ ] Copy the `database_id` from output → paste into `wrangler.toml`
- [ ] Apply migrations locally: `bun run db:migrate:local`
- [ ] Verify: `wrangler d1 execute yun-perfume-db --local --command "SELECT name FROM sqlite_master WHERE type='table'"`

1.7 Seed the database

- [ ] Create `scripts/seed.ts` using data from `src/lib/fragrances.ts` as source of truth:

```ts
// Seed these 4 products exactly matching fragrances.ts names/descriptions
const PRODUCTS = [
  { slug: 'mogra-noir',     name: 'Mogra Noir',    price: 480000, scentFamily: 'Fl
  { slug: 'oud-vara',       name: 'Oud Vārā',      price: 680000, scentFamily: 'Wo
  { slug: 'sandal-velvet',  name: 'Sandal Velvet', price: 540000, scentFamily: 'Wo
  { slug: 'chai-atelier',   name: 'Chai Atelier',  price: 420000, scentFamily: 'Go
];
// Also seed: 1 collection, 1 ADMIN user (your email)
```

- [ ] Run: `bun run db:seed`
- [ ] Verify: `wrangler d1 execute yun-perfume-db --local --command "SELECT name, price FROM products"`

1.8 Update admin dashboard to use Drizzle

- [ ] Open `src/app/admin/page.tsx`
- [ ] Add `export const runtime = 'edge'`
- [ ] Replace all Prisma imports with Drizzle:

```ts
import { getRequestContext } from '@cloudflare/next-on-pages';
import { getDb, products, orders } from '@/db';
import { count, sum, eq } from 'drizzle-orm';
```

- [ ] Replace all Prisma queries with Drizzle equivalents
- [ ] Preserve existing UI — just swap the data source

PHASE 2 — Cloudflare Services Setup

2.1 D1 (done in Phase 1.6)

- [ ] Confirmed: `database_id` in `wrangler.toml`
- [ ] Confirmed: migrations applied locally

2.2 R2 bucket

- [ ] Run: `wrangler r2 bucket create yun-perfume-images`

> Matches existing `bucket_name` in old config — consistent

- [ ] In Cloudflare dashboard: R2 > `yun-perfume-images` > Settings > Public Access > Enable
- [ ] Copy the public bucket URL (format: `https://pub-XXXX.r2.dev`)
- [ ] Set in `.env.local`: `NEXT_PUBLIC_R2_PUBLIC_URL=https://pub-XXXX.r2.dev`
- [ ] Optional: configure `assets.yunperfume.com` CNAME in Cloudflare DNS for a cleaner URL

2.3 KV namespace

- [ ] Run: `wrangler kv namespace create SESSIONS`
- [ ] Run: `wrangler kv namespace create SESSIONS --preview`
- [ ] Paste both IDs into `wrangler.toml` (replaces `"change-me-later"` from old config)

2.4 Store production secrets

- [ ] Run each and paste value when prompted:

```bash
```

```
wrangler secret put RAZORPAY_KEY_ID
wrangler secret put RAZORPAY_KEY_SECRET
wrangler secret put RAZORPAY_WEBHOOK_SECRET
wrangler secret put NEXT_PUBLIC_RAZORPAY_KEY_ID
wrangler secret put RESEND_API_KEY
wrangler secret put JWT_SECRET
wrangler secret put NEXT_PUBLIC_APP_URL
wrangler secret put NEXT_PUBLIC_R2_PUBLIC_URL
```

- [ ] Verify: Cloudflare dashboard > Pages > yun-perfume > Settings > Environment Variables

2.5 Connect GitHub to Cloudflare Pages

- [ ] Dashboard > Pages > Create project > Connect to Git > select `yun-perfume` repo
- [ ] Build command: `bun run pages:build`
- [ ] Output directory: `.vercel/output/static`
- [ ] Node version: `20`
- [ ] Save and trigger first build

2.6 Custom domain

- [ ] Pages project > Custom domains > `yunperfume.com`
- [ ] Confirm SSL Active (green)

PHASE 3 — Auth: Magic Link (Fix End-to-End)

> Current state: `auth-store.tsx` simulates auth in the browser entirely. Token stored in `sessionStorage`. API route exists but is never called. No server-side token storage = can never be verified.

3.1 Create auth utilities

- [ ] Create `src/lib/auth.ts`:

```ts
```

```ts
import { SignJWT, jwtVerify } from 'jose';
import { nanoid } from 'nanoid';

const getSecret = () => new TextEncoder().encode(process.env.JWT_SECRET!);

export async function createSessionToken(userId: string, role: string): Promise<st
  return new SignJWT({ userId, role })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('30d')
    .sign(getSecret());
}

export async function verifySessionToken(token: string) {
  try {
    const { payload } = await jwtVerify(token, getSecret());
    return payload as { userId: string; role: string };
  } catch {
    return null;
  }
}

export function generateMagicToken(): string {
  return nanoid(40);
}

export function getMagicTokenExpiry(): string {
  const d = new Date();
  d.setMinutes(d.getMinutes() + 15);
  return d.toISOString();
}
```

3.2 Create session helper

- [ ] Create `src/lib/session.ts`:

```ts
```

```
import { cookies } from 'next/headers';
import { verifySessionToken } from './auth';

export async function getSession() {
  const cookieStore = await cookies();
  const token = cookieStore.get('yun_session')?.value;
  if (!token) return null;
  return verifySessionToken(token);
}

export async function requireSession() {
  const session = await getSession();
  if (!session) return null;
  return session;
}

export async function requireAdmin() {
  const session = await getSession();
  if (!session || session.role !== 'ADMIN') return null;
  return session;
}
```

3.3 Rewrite `POST /api/auth/magic-link/route.ts`

- [ ] Open `src/app/api/auth/magic-link/route.ts` — **replace entirely**:
  - Add `export const runtime = 'edge'`
  - Validate email with Zod
  - Rate limit via KV: max 3 requests per email per 10 minutes
  - Look up or create user in D1
  - Generate token with `generateMagicToken()`
  - **Store token in D1 `magic_tokens` table** — this is the key fix that was missing
  - Send email via Resend with magic link URL: `${APP_URL}/api/auth/verify?token=${token}`
  - Always return `{ success: true }` regardless of whether email exists

3.4 Create `GET /api/auth/verify/route.ts` (new file)

- [ ] Create `src/app/api/auth/verify/route.ts`:
  - Read `?token=` from query params
  - Look up token in D1 `magic_tokens` — must exist, unused, not expired
  - If invalid: redirect to `/login?error=invalid`
  - If expired: redirect to `/login?error=expired`

- Mark token `usedAt = now()` immediately
- Get user from D1, update `lastLoginAt`
- Create JWT with `createSessionToken(userId, role)`
- Set `yun_session` cookie: `HttpOnly: true`, `Secure: true`, `SameSite: lax`, 30 days
- Redirect: ADMIN role → `/admin`, USER role → `/profile`

3.5 Create `POST /api/auth/signout/route.ts` (new file)

☐ Create `src/app/api/auth/signout/route.ts`:

- Delete `yun_session` cookie
- Return `{ success: true }`

3.6 Update `src/lib/auth-store.tsx`

☐ Replace `sendMagicLink()` — remove all `sessionStorage`/`localStorage` token logic:

```ts
const sendMagicLink = async (email: string) => {
  const res = await fetch('/api/auth/magic-link', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email }),
  });
  if (!res.ok) throw new Error('Failed to send magic link');
};
```

☐ Remove `verify()` function entirely — verification is now handled server-side via the `/api/auth/verify` redirect
☐ Keep the `user` state and `isLoggedIn` — these can be hydrated from session on page load if needed

3.7 Update `src/app/login/page.tsx`

☐ Step 1: Email input → calls `sendMagicLink(email)` → show "Check your email" state
☐ Remove the "enter the code" / OTP input step — the link in the email IS the verification
☐ Handle error states from query params:

- `?error=invalid` → show "This link is invalid or has already been used."
- `?error=expired` → show "This link has expired. Request a new one."

3.8 Protect routes with middleware

☐ Create `src/middleware.ts`:

```ts
```

```
import { NextRequest, NextResponse } from 'next/server';
import { verifySessionToken } from '@/lib/auth';

export const runtime = 'edge';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  if (pathname.startsWith('/admin')) {
    const token = request.cookies.get('yun_session')?.value;
    if (!token) return NextResponse.redirect(new URL('/login', request.url));
    const session = await verifySessionToken(token);
    if (!session || session.role !== 'ADMIN') {
      return NextResponse.redirect(new URL('/login', request.url));
    }
  }

  if (pathname.startsWith('/profile')) {
    const token = request.cookies.get('yun_session')?.value;
    if (!token) return NextResponse.redirect(new URL('/login', request.url));
    const session = await verifySessionToken(token);
    if (!session) return NextResponse.redirect(new URL('/login', request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/admin/:path*', '/profile/:path*'],
};
```

PHASE 4 — Products: Static → Database

> Current state: `shop/page.tsx` and `shop/[slug]/page.tsx` read from `src/lib/fragrances.ts`. Products are not in D1. Admin dashboard shows 0 products.

4.1 Keep `fragrances.ts` as reference

- [ ] Do NOT delete `src/lib/fragrances.ts`
- [ ] Add comment at top: `// Reference only — live data served from D1. See scripts/seed.ts`

4.2 Create Products API

- [ ] Create `src/app/api/products/route.ts`:

- `GET`: return all active products from D1
- Use ISR: `next: { revalidate: 60 }`

☐ Create `src/app/api/products/[slug]/route.ts`:

- `GET`: return single product by slug
- Return 404 if not found or inactive

4.3 Update `src/app/shop/page.tsx`

☐ Remove `import { fragrances } from '@/lib/fragrances'`
☐ Remove `import { useCart }` (line 5 bug — already tracked in Phase 0.8)
☐ Fetch products server-side from `GET /api/products`
☐ Map D1 product fields to existing UI component props:

- `product.price` (paise) → `formatINR(product.price / 100)` for display
- `product.scentNotes` (JSON string) → `JSON.parse(product.scentNotes)` for tags
- `product.imageUrl` → existing image slot

☐ Preserve all existing visual styling, staggered layout, GSAP animations

4.4 Update `src/app/shop/[slug]/page.tsx`

☐ Remove static fragrance lookup
☐ Fetch product from `GET /api/products/${params.slug}`
☐ Call `notFound()` if API returns 404
☐ Parse `scentNotes` and `images` JSON strings before rendering
☐ Price: `formatINR(product.price / 100)` — use existing `utils.ts` helper
☐ Add `generateStaticParams()`:

```ts
export async function generateStaticParams() {
  return ['mogra-noir', 'oud-vara', 'sandal-velvet', 'chai-atelier'].map(s => ({ s
}
```

☐ Preserve all existing UI: size selector, Add to bag, GSAP entrance, related products section

4.5 Update cart item price handling

☐ Open `src/lib/cart-store.tsx`
☐ When adding to cart from product page: confirm `price` is stored in INR (rupees)
☐ DB gives paise → divide by 100 before adding to cart: `price: product.price / 100`
☐ Cart totals, subtotal display all already work in INR — no other changes needed

---

PHASE 5 — Email Integration (Resend)

> Current state: `RESEND_API_KEY` exists in `.env`. The magic link API route has Resend imported but the client never calls the route. Contact form never sends anything.

5.1 Resend client

- [ ] Create `src/lib/resend.ts`:

```ts
import { Resend } from 'resend';
export const resend = new Resend(process.env.RESEND_API_KEY!);
export const FROM = 'YUN Perfume <orders@yunperfume.com>';
```

5.2 Magic link email template

- [ ] Create `src/lib/emails/magic-link.ts`:
  - From: `YUN Perfume <orders@yunperfume.com>`
  - Subject: `Your access link — YUN Perfume`
  - Verify URL: `${process.env.NEXT_PUBLIC_APP_URL}/api/auth/verify?token=${token}`
  - Design: dark background `#0a0a14`, ivory text, cobalt CTA button matching brand
  - CTA text: `ENTER THE ATELIER →`
  - Footer: "This link expires in 15 minutes and can only be used once."
  - Use inline styles only (email client compatibility)

5.3 Order confirmation email template

- [ ] Create `src/lib/emails/order-confirmation.ts`:
  - Subject: `Order confirmed — ${orderId} | YUN Perfume`
  - Header: "YOUR ORDER HAS BEEN CAPTURED."
  - Items table: name, size/concentration, qty, price in INR
  - Shipping address block
  - Total breakdown in INR
  - Footer: "Encased in signature packaging. Dispatched within 3–5 business days from Pune."
  - Export as `sendOrderConfirmationEmail(order, items)` function

5.4 Fix contact form

- [ ] Open `src/app/contact/page.tsx`
- [ ] Replace `setSubmitted(true)` in `onSubmit` with real fetch call to `POST /api/contact`
- [ ] Create `src/app/api/contact/route.ts`:
  - `export const runtime = 'edge'`

- Validate: name, email, message with Zod
- Send via Resend to `hello@yunperfume.com`
- Set `replyTo: senderEmail`
- Return `{ success: true }`

PHASE 6 — Razorpay: Full Payment Integration

> Current state: `api/payment/create-order/route.ts` exists and calls Razorpay SDK. The checkout page NEVER calls it — it does a fake `setTimeout(1400)` and clears the cart. No webhook handler exists anywhere.

6.1 Razorpay helper

☐ Create `src/lib/razorpay.ts`:

```ts
import Razorpay from 'razorpay';
import crypto from 'crypto';

export const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID!,
  key_secret: process.env.RAZORPAY_KEY_SECRET!,
});

// Used for webhook verification (uses WEBHOOK_SECRET, not API secret)
export function verifyWebhookSignature(rawBody: string, signature: string): boolea
  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(rawBody)
    .digest('hex');
  return expected === signature;
}
```

6.2 Rewrite `POST /api/payment/create-order/route.ts`

☐ Open existing file — **replace entirely**:

- `export const runtime = 'edge'`
- Validate request body with Zod: `{ items, shippingAddress, email }`
- Get session from cookie (optional — guest checkout allowed if no session)
- **Fetch product prices from D1 — NEVER trust client-sent prices**
- Validate each product: exists, active, has sufficient stock

- Calculate server-side: subtotal, GST 18%, shipping (free ≥ ₹1500), total — all in paise
- Create Razorpay order via SDK
- Save draft order to D1 with `status: 'PENDING'`
- Save order items to D1
- Return: `{ orderId, razorpayOrderId, amount, currency, keyId }`

6.3 Create `POST /api/payment/webhook/route.ts` (new file — critical missing piece)

- [ ] Create `src/app/api/payment/webhook/route.ts`:
  - `export const runtime = 'edge'`
  - Read raw body with `req.text()` (must NOT use `req.json()` — signature verification requires raw body)
  - **Verify Razorpay signature immediately — return 400 if invalid**
  - Handle `payment.captured`:
    - Find order by `razorpayOrderId` in D1
    - **Idempotency check**: if order already `PAID`, return 200 and do nothing
    - Update order: `status = 'PAID'`, set `razorpayPaymentId`, set `paidAt`
    - Decrement stock for each order item: `stock = stock - quantity`
    - Send order confirmation email via Resend
  - Handle `payment.failed`:
    - Update order `status = 'FAILED'`
  - **Always return 200 to Razorpay** — non-200 causes Razorpay to retry indefinitely

6.4 Fix `src/app/checkout/page.tsx`

- [ ] Add `'use client'` at top (page needs browser Razorpay SDK)
- [ ] Find the fake payment handler — the `setTimeout(1400)` block
- [ ] **Delete the entire fake setTimeout block**
- [ ] Replace with real Razorpay flow:
  1. Collect shipping address from form fields (name, line1, city, state, pincode, phone)
  2. Collect email: from auth state if logged in, or add an email field to the form
  3. On "PAY" click: call `POST /api/payment/create-order`
  4. Load Razorpay script: `https://checkout.razorpay.com/v1/checkout.js` (dynamic script tag)
  5. Open Razorpay modal with `order_id`, `amount`, `key`, theme color `#4a6fa5`
  6. `handler` callback (payment success): clear cart → `router.push('/order/${orderId}')`
  7. `modal.ondismiss`: set loading to false, show "Payment was cancelled"
- [ ] Add loading state to PAY button (spinner while API call + modal opening)

- [ ] Add error state display for API errors (out of stock, etc.)

6.5 Create order success page

- [ ] Create `src/app/order/[id]/page.tsx`:
  - `'use client'` — needs polling
  - Poll `GET /api/orders/[id]/status` every 2 seconds
  - PENDING state: "Confirming your order..." with loading indicator (use brand CSS classes)
  - PAID state: show order ID, items, total in INR, shipping address, dispatch timeline
  - FAILED state: error message + "Return to cart" → `/shop`
  - Stop polling after 30 seconds — show "Contact us at hello@yunperfume.com if payment was deducted"
  - CTA after PAID: "CONTINUE EXPLORING →" → `/shop`

6.6 Create order status API

- [ ] Create `src/app/api/orders/[id]/status/route.ts`:
  - `export const runtime = 'edge'`
  - `GET`: return `{ status }` for the given order ID
  - No auth required (order ID is non-guessable due to nanoid suffix)

6.7 Configure Razorpay webhook

- [ ] After first production deploy, go to Razorpay dashboard > Settings > Webhooks
- [ ] Add webhook URL: `https://yunperfume.com/api/payment/webhook`
- [ ] Enable events: `payment.captured`, `payment.failed`
- [ ] Copy the webhook secret shown
- [ ] Store it: `wrangler secret put RAZORPAY_WEBHOOK_SECRET`

PHASE 7 — Profile Page (Real Session)

> Current state: hardcoded `collector@example.com`, no real session mechanism.

7.1 Create orders list API

- [ ] Create `src/app/api/orders/route.ts`:
  - `export const runtime = 'edge'`
  - Read session from `yun_session` cookie
  - Return 401 if no valid session
  - Return all orders for `session.userId` from D1, ordered by `createdAt DESC`

- [ ] Convert to Server Component
- [ ] Call `requireSession()` — redirect to `/login` if null
- [ ] Fetch real user orders from D1
- [ ] Display real user email from session
- [ ] Order history: ID, date, status badge, total in INR, items count
- [ ] Sign out button: `POST /api/auth/signout` → redirect to `/`
- [ ] Preserve all existing visual styling

PHASE 8 — Admin Panel (Complete + Protect)

> Current state: `/admin` has NO auth. All sub-pages (`/admin/products`, `/admin/orders`, `/admin/products/new`, `/admin/collections/new`) are 404.

8.1 Verify middleware protection (from Phase 3.8)

- [ ] Test: `/admin` without session → redirects to `/login` ✓
- [ ] Test: `/admin` with USER role → redirects to `/login` ✓
- [ ] Test: `/admin` with ADMIN role → loads dashboard ✓

8.2 Admin layout

- [ ] Open `src/app/admin/layout.tsx`
- [ ] Add server-side double-check:

```ts
import { requireAdmin } from '@/lib/session';
import { redirect } from 'next/navigation';

export default async function AdminLayout({ children }) {
  const session = await requireAdmin();
  if (!session) redirect('/login');
  return <>{children}</>;
}
```

- [ ] Add sidebar navigation: Dashboard, Products, Orders, Collections

8.3 Admin API auth helper

- [ ] Create `src/lib/admin-auth.ts`:

```ts
```

```
import { NextRequest } from 'next/server';
import { verifySessionToken } from './auth';

export async function requireAdminFromRequest(req: NextRequest) {
  const token = req.cookies.get('yun_session')?.value;
  if (!token) return null;
  const session = await verifySessionToken(token);
  if (!session || session.role !== 'ADMIN') return null;
  return session;
}
```

- [ ] Import and call at the top of every admin API route

8.4 Products list page

- [ ] Create `src/app/admin/products/page.tsx`:
  - Table: thumbnail, name, price in INR, stock, active status, edit link
  - Low stock highlight: stock ⩽ 5
  - "ADD NEW PRODUCT" button → `/admin/products/new`

8.5 Product create/edit form

- [ ] Create `src/app/admin/products/new/page.tsx`
- [ ] Create `src/app/admin/products/[id]/edit/page.tsx`
- [ ] Shared `ProductForm` component with fields:
  - Name, slug (auto-generated, editable), tagline, description
  - Price in INR (store as paise: `Math.round(priceINR * 100)`)
  - Stock, active toggle
  - Scent family, scent notes (tag input)
  - Concentration (dropdown), size
  - Image upload (calls `POST /api/upload` → stores R2 URL)
- [ ] Create `src/app/api/admin/products/route.ts` (POST — create new product, admin only)
- [ ] Create `src/app/api/admin/products/[id]/route.ts` (PATCH — update, admin only)

8.6 Orders list page

- [ ] Create `src/app/admin/orders/page.tsx`:
  - Table: order ID, customer email, date, status badge, total in INR, view link
  - Status filter: ALL / PENDING / PAID / SHIPPED / DELIVERED / FAILED
  - Status badge colours: PENDING=amber, PAID=green, SHIPPED=blue, FAILED=red

8.7 Order detail page

- [ ] Create `src/app/admin/orders/[id]/page.tsx`:
  - Full order: customer email, items with quantities, total breakdown, shipping address
  - Status update: dropdown to change to SHIPPED / DELIVERED / CANCELLED
- [ ] Create `src/app/api/admin/orders/[id]/route.ts` (PATCH — update status, admin only)

8.8 Collections page

- [ ] Create `src/app/admin/collections/page.tsx`:
  - List all collections with product count
  - Create new collection form: name, slug, description
- [ ] Create `src/app/api/admin/collections/route.ts` (GET + POST, admin only)

PHASE 9 — R2 Image Storage

> Current state: `IMAGES` R2 binding declared but unused. Product images are `src/assets/` static imports. `product.imageUrl` in DB is null.

9.1 Image upload API

- [ ] Create `src/app/api/upload/route.ts`:
  - `export const runtime = 'edge'`
  - Admin only — call `requireAdminFromRequest()`
  - Accept `multipart/form-data` with `file` field
  - Validate: max 5MB, types: `image/jpeg`, `image/png`, `image/webp` only
  - Generate key: `products/${nanoid()}.${ext}`
  - Upload to R2: `env.IMAGES.put(key, buffer, { httpMetadata: { contentType } })`
  - Return `{ url: '${R2_PUBLIC_URL}/${key}', key }`

9.2 Upload existing product assets to R2

- [ ] Identify actual image filenames in `src/assets/` for each product
- [ ] Upload to R2:

```bash
wrangler r2 object put yun-perfume-images/products/mogra-noir.jpg --file src/asset
wrangler r2 object put yun-perfume-images/products/oud-vara.jpg --file src/assets/
wrangler r2 object put yun-perfume-images/products/sandal-velvet.jpg --file src/as
wrangler r2 object put yun-perfume-images/products/chai-atelier.jpg --file src/ass
```

- [ ] Update `scripts/seed.ts` to include actual R2 URLs in `imageUrl` field

9.3 R2 URL helper

- [ ] Create `src/lib/r2.ts`:

```ts
export function getR2Url(key: string): string {
  return `${process.env.NEXT_PUBLIC_R2_PUBLIC_URL}/${key}`;
}
```

PHASE 10 — Polish & Edge Cases

10.1 Discovery Set page

- [ ] Open `src/app/shop/discovery-set/page.tsx`
- [ ] Wire "Add to bag" to cart store
- [ ] Ensure discovery set product exists in DB seed with `isDiscoverySet: true`
- [ ] Price and product name should come from DB, not hardcoded

10.2 404 and error pages

- [ ] Create `src/app/not-found.tsx` — brand-appropriate, dark cobalt, `.h-display` typography
- [ ] Create `src/app/error.tsx` — with retry button, same aesthetic

10.3 Loading states

- [ ] Create `src/app/shop/loading.tsx` — skeleton matching product grid
- [ ] Create `src/app/admin/loading.tsx` — minimal spinner

10.4 SEO additions

- [ ] Add `noindex` meta to `/order/[id]`, `/admin/*`, `/login`
- [ ] Create `src/app/sitemap.ts` with all 4 product URLs
- [ ] Create `src/app/robots.ts` — disallow `/admin`, `/api`, `/order`

PHASE 11 — Security Checklist

- [ ] Every `src/app/api/` route has `export const runtime = 'edge'`
- [ ] Every admin API route calls `requireAdminFromRequest(req)` and returns 403 if null
- [ ] Webhook verifies Razorpay signature — no exceptions — uses `RAZORPAY_WEBHOOK_SECRET`
- [ ] Order amounts calculated server-side only — client cart prices never trusted
- [ ] Stock only decremented in webhook handler after `payment.captured` confirmed

- [ ] Webhook is idempotent — handles duplicate `payment.captured` events gracefully
- [ ] Magic tokens single-use — marked `usedAt` immediately on first verification
- [ ] `JWT_SECRET` is ≥ 64 characters — generated with `openssl rand -base64 64`
- [ ] `yun_session` cookie: `HttpOnly`, `Secure`, `SameSite: lax`
- [ ] No secrets in `wrangler.toml` — all via `wrangler secret put`
- [ ] `.env.local` confirmed in `.gitignore`
- [ ] Rate limiting on magic link endpoint (KV counter, 3/email/10min)
- [ ] File upload validates MIME type AND file size before R2
- [ ] Zod validation on ALL API inputs

PHASE 12 — Testing & QA

12.1 Auth

- [ ] Enter email on `/login` → real email arrives within 60 seconds
- [ ] Click magic link → correct redirect (USER → `/profile`, ADMIN → `/admin`)
- [ ] Use link twice → error shown
- [ ] Wait 15 min, use link → expired error shown
- [ ] `/admin` without session → `/login`
- [ ] `/admin` as USER → `/login`
- [ ] `/profile` without session → `/login`

12.2 Shop & cart

- [ ] All 4 products load from D1 (not from `fragrances.ts`)
- [ ] Product detail pages load with correct DB data
- [ ] Add to cart → drawer opens, header count updates
- [ ] Cart persists on refresh
- [ ] Prices display in INR (not paise)

12.3 Payment flow (test mode)

- [ ] "PAY" button calls real API — no setTimeout
- [ ] Razorpay modal opens with correct amount
- [ ] Test success: card `4111 1111 1111 1111`, any future expiry, any CVV
- [ ] Modal closes → `/order/[id]` shows "Confirming..."
- [ ] Webhook fires → order becomes PAID
- [ ] Order success page updates to show confirmation
- [ ] Order confirmation email received
- [ ] Order in admin panel with PAID status
- [ ] Product stock decremented in D1

12.4 Payment failure

- [ ] Test failure: card `4000 0000 0000 0002`
- [ ] No stock decrement, no email
- [ ] `/order/[id]` shows failure state with retry

12.5 Admin

- [ ] Auth blocking works (Phase 12.1)
- [ ] Dashboard stats are accurate
- [ ] Create product → appears on `/shop`
- [ ] Edit price → updates on storefront
- [ ] Set stock 0 → out-of-stock state
- [ ] Orders list shows all test orders
- [ ] Update order to SHIPPED → status updates

12.6 Contact form

- [ ] Submit → email arrives at `hello@yunperfume.com`
- [ ] Success state shown

12.7 Mobile (375px + 390px)

- [ ] All pages render correctly
- [ ] Cart drawer usable
- [ ] Checkout form usable
- [ ] Razorpay modal works on iOS Safari and Android Chrome

PHASE 13 — Production Deployment

13.1 Pre-deploy

- [ ] Apply migrations to production D1: `bun run db:migrate:remote`
- [ ] Seed production D1 (run seed script with `--remote` flag)
- [ ] Upload product images to production R2 bucket
- [ ] All Cloudflare secrets set (`wrangler secret list` to verify all 8 are present)
- [ ] Switch to live Razorpay keys:

```bash
wrangler secret put RAZORPAY_KEY_ID            # rzp_live_...
wrangler secret put RAZORPAY_KEY_SECRET
wrangler secret put NEXT_PUBLIC_RAZORPAY_KEY_ID
```

- [ ] Razorpay KYC fully approved

- [ ] Resend domain `yunperfume.com` verified and active

13.2 Deploy

- [ ] Push to `main` → auto-deploys via GitHub integration
- [ ] Monitor Cloudflare Pages build log
- [ ] Build must complete with 0 errors

13.3 Configure live Razorpay webhook

- [ ] Add webhook: `https://yunperfume.com/api/payment/webhook`
- [ ] Enable: `payment.captured`, `payment.failed`
- [ ] Copy webhook secret → `wrangler secret put RAZORPAY_WEBHOOK_SECRET`
- [ ] Re-deploy to pick up new secret: push empty commit or trigger manual deploy

13.4 Post-deploy smoke test

- [ ] `https://yunperfume.com` loads with GSAP animations intact
- [ ] All 4 products on `/shop`
- [ ] Complete a real ₹1 live transaction
- [ ] Order confirmation email received
- [ ] Admin panel shows the live order
- [ ] SSL is green on all pages

13.5 Monitoring

- [ ] Cloudflare Analytics enabled (free — dashboard toggle)
- [ ] Cloudflare D1 query metrics visible
- [ ] Razorpay webhook alert email configured
- [ ] Cloudflare Pages build failure notifications enabled

APPENDIX — Quick Reference

Razorpay test credentials

	Value
Success card	`4111 1111 1111 1111`
Failure card	`4000 0000 0000 0002`
Expiry	Any future MM/YY
CVV	Any 3 digits

	Value
Success UPI	`success@razorpay`
Failure UPI	`failure@razorpay`

Order status flow

```
PENDING → PAID       webhook: payment.captured
PENDING → FAILED     webhook: payment.failed
PAID    → SHIPPED    admin panel action
SHIPPED → DELIVERED  admin panel action
ANY     → CANCELLED  admin panel action
```

Price rule (critical — must be consistent everywhere)

```
D1 stores paise:      ₹4,800 = 480000  (integer, no decimals)
UI displays INR:      480000 / 100 = ₹4,800  (use formatINR from utils.ts)
Razorpay takes paise: send 480000 directly from DB — no conversion needed
Cart stores INR:      product.price / 100 before adding to cart
```

Files changed — at a glance

File	Action
`wrangler.jsonc`	DELETE → replaced by `wrangler.toml`
`prisma/` folder	DELETE → replaced by `src/db/` + Drizzle
`src/lib/db.ts`	DELETE → replaced by `src/db/index.ts`
`src/app/api/auth/magic-link/route.ts`	REWRITE — now stores token in D1
`src/app/api/payment/create-order/route.ts`	REWRITE — validates stock, uses D1
`src/app/api/payment/webhook/route.ts`	CREATE — was missing entirely
`src/app/api/auth/verify/route.ts`	CREATE — was missing entirely
`src/app/api/auth/signout/route.ts`	CREATE — was missing entirely
`src/app/api/contact/route.ts`	CREATE — was missing entirely
`src/app/api/orders/route.ts`	CREATE — was missing entirely
`src/app/api/orders/[id]/status/route.ts`	CREATE — was missing entirely

File	Action
`src/app/order/[id]/page.tsx`	CREATE — order success page
`src/app/checkout/page.tsx`	FIX — delete fake setTimeout, wire Razorpay
`src/app/login/page.tsx`	FIX — remove sessionStorage, handle errors
`src/app/profile/page.tsx`	FIX — remove hardcoded email, use real session
`src/app/shop/page.tsx`	FIX — remove useCart import, switch to D1
`src/app/shop/[slug]/page.tsx`	FIX — switch from fragrances.ts to D1
`src/app/contact/page.tsx`	FIX — wire form to /api/contact
`src/lib/auth-store.tsx`	FIX — wire sendMagicLink to real API
`src/middleware.ts`	CREATE — route protection
`src/lib/auth.ts`	CREATE — JWT + token helpers
`src/lib/session.ts`	CREATE — server component session helper
`src/lib/razorpay.ts`	CREATE — Razorpay client + sig verification
`src/lib/resend.ts`	CREATE — Resend client
`src/lib/admin-auth.ts`	CREATE — admin API auth helper
`src/lib/r2.ts`	CREATE — R2 URL helper
`src/db/schema.ts`	CREATE — Drizzle schema
`src/db/index.ts`	CREATE — Drizzle D1 connection

Env variables — final complete list

Variable	Exposed to browser?	Source
`RAZORPAY_KEY_ID`	No	Razorpay dashboard
`RAZORPAY_KEY_SECRET`	No	Razorpay dashboard
`RAZORPAY_WEBHOOK_SECRET`	No	Razorpay dashboard > Webhooks
`NEXT_PUBLIC_RAZORPAY_KEY_ID`	Yes	Same as KEY_ID

Variable	Exposed to browser?	Source
`RESEND_API_KEY`	No	Resend dashboard
`JWT_SECRET`	No	`openssl rand -base64 64`
`NEXT_PUBLIC_APP_URL`	Yes	`https://yunperfume.com`
`NEXT_PUBLIC_R2_PUBLIC_URL`	Yes	R2 bucket public URL